

Hackathon MedConnect — Guide complet pour les équipes

1. Bienvenue 🦷

Bienvenue au Hackathon MedConnect ! Pendant cet événement, **12 équipes** auront un accès direct à notre moteur d'analyse de radiographies dentaires basé sur l'intelligence artificielle. C'est le même moteur qui fait tourner notre produit en production : il identifie automatiquement les dents, les caries, les implants, les prothèses, les pathologies osseuses et bien plus encore — et il vous renvoie tout cela sous forme de JSON structuré, avec coordonnées de bounding box pour superposer vos visualisations.

Votre mission : **construire quelque chose de remarquable** avec ces super-pouvoirs en 48–72 heures.

L'API que vous allez utiliser dessert quotidiennement de vraies cliniques dentaires dans plusieurs pays. Vous travaillez avec un outil professionnel, pas un jouet — alors voyez grand.

2. Ce que fait l'API

L'API ThakaaMed prend en entrée une **radiographie dentaire** (image JPEG/PNG/DICOM converti) et renvoie une analyse complète comprenant :

Type de radiographie	Acceptée	Modèles disponibles
Panoramique	✓	Détection complète : dents, caries, restaurations, implants, prothèses, pathologies péri-apicales, niveau osseux, calculs, sinus, condyles
Périapicale	✓	Détection focale haute résolution sur 1–4 dents
Bitewing (rétro-coronaire)	✓	Caries inter-proximales, restaurations, niveau osseux
Céphalométrique latérale	✓	Repères céphalométriques, calibrage, plan de Frankfort

Pour chaque détection, l'API renvoie :

- Le **type de structure ou pathologie** détecté (dent par numérotation FDI, type de carie, type de restauration, etc.)
- Un **score de confiance** (0–1)
- Les **coordonnées de bounding box** (rectangle englobant)
- Les **coordonnées polygonales** (forme exacte de la détection — utile pour les overlays précis)
- La **dent associée** (par exemple « dent 36, carie occlusale »)

Le moteur exécute **191 modèles d'IA** différents par radiographie panoramique. C'est sérieux.

3. Vos identifiants

Chaque équipe reçoit :

- Une **clé API** (**api_key**) — 32 caractères hexadécimaux majuscules.
- Un **code d'établissement** (**facility_code**) — requis sur chaque appel.
- **1 000 jetons** (1 jeton = 1 analyse de radiographie, peu importe le type).
- Une **durée de validité de 30 jours** (les clés expirent automatiquement le **2026-06-08**).

4. Endpoint et appel de l'API

URL principale (recommandée — version 2.3)

POST <https://aiv4.thakaamed.com/api/v2.3/en/analyze/radiography/>

Le segment **/xx/** dans l'URL = langue de la réponse

Le segment **/en/** dans l'URL contrôle **uniquement la langue des libellés textuels** renvoyés dans la réponse JSON (**label_name**, descriptions, messages d'erreur). **La détection IA est strictement identique dans toutes les langues** — mêmes modèles, mêmes coordonnées, mêmes scores de confiance, juste les textes traduits.

Code	Langue	Exemple d'URL
en	Anglais	.../api/v2.3/en/analyze/radiography/
fr	Français	.../api/v2.3/fr/analyze/radiography/
ar	Arabe	.../api/v2.3/ar/analyze/radiography/
tr	Turc	.../api/v2.3/tr/analyze/radiography/
de	Allemand	.../api/v2.3/de/analyze/radiography/
nl	Néerlandais	.../api/v2.3/nl/analyze/radiography/
pt	Portugais	.../api/v2.3/pt/analyze/radiography/
ru	Russe	.../api/v2.3/ru/analyze/radiography/
es	Espagnol	.../api/v2.3/es/analyze/radiography/
uz	Ouzbek	.../api/v2.3/uz/analyze/radiography/

💡 **Idée de cas d'usage multilingue** : pour un cabinet ou une app servant plusieurs communautés (par exemple au Maroc : français + arabe + anglais), faites **un seul POST**, mémorisez le **slug**, puis faites **plusieurs GET** sur des URLs de langues différentes — vous obtiendrez le même résultat, traduit. Idéal pour le Défi 1 (Explicateur pour Patients) ou le Défi 7 (Téléconsultation).

```
# Soumission une seule fois (consomme 1 jeton)

slug = submit_image('panoramique.jpg') # POST sur /en/

# Récupération dans 3 langues – gratuit, le même résultat traduit

res_fr = requests.get('https://aiv4.thakaamed.com/api/v2.3/fr/analyze/radiography/',
params={'id': slug}).json()

res_ar = requests.get('https://aiv4.thakaamed.com/api/v2.3/ar/analyze/radiography/',
params={'id': slug}).json()

res_en = requests.get('https://aiv4.thakaamed.com/api/v2.3/en/analyze/radiography/',
params={'id': slug}).json()
```

Pattern asynchrone : POST puis GET

L'analyse d'une radiographie prend entre 5 et 30 secondes selon le type d'image et la charge serveur. L'API utilise donc un pattern **asynchrone en deux temps** :

Étape 1 — Soumettre l'image (POST)

```
curl -X POST 'https://aiv4.thakaamed.com/api/v2.3/fr/analyze/radiography/' \
-F 'api_key=VOTRE_CLE_API' \
-F 'facility_code=VOTRE_CODE_ETABLISSEMENT' \
-F 'image=@chemin/vers/radio.jpg'
```

Réponse immédiate (200 OK) :

```
{
  "status": true,
  "id": "9941c5cf-6e4f-4f02-9425-...",
  "results": "https://aiv4.thakaamed.com/api/v2.3/fr/analyze/radiography/?id=9941c5cf-...",
  "Warning__Message!!!": "No Warning."
}
```

- **status: true** ici signifie « **soumission acceptée** » — pas que l'analyse est terminée.
- **id** est votre **slug d'analyse**. Conservez-le pour le polling.

- **results** est l'**URL prête à l'emploi pour le GET** (pratique).
- **Warning__Message!!!** (sic) signale par exemple une image trop petite — l'analyse continue quand même.

Étape 2 — Récupérer le résultat (GET)

```
curl 'https://aiv4.thakaamed.com/api/v2.3/fr/analyze/radiography/?id=9941c5cf-...'
```

L'analyse se termine en général en **5 à 30 secondes** (souvent <10 s sur l'infrastructure de hackathon). Le signal de fin est **is_done: true** dans la réponse GET — **ce n'est PAS le champ `status`**, qui est uniquement présent dans la réponse POST.

```
{
  "id": "...",
  "is_done": true,           // ← votre signal de fin
  "error_status": false,
  "message": "Analysis completed successfully.",
  "results": { /* ... voir §5 ... */ }
}
```

Mettez en place une boucle de polling avec un délai de **2 à 5 secondes** entre les requêtes.

Exemple Python (client minimal)

```
import requests

import time

API_KEY = 'VOTRE_CLE_API'

FACILITY = 'VOTRE_CODE_ETABLISSEMENT'

BASE = 'https://aiv4.thakaamed.com/api/v2.3/fr/analyze/radiography/'

def analyze(image_path):
    # Soumission
    with open(image_path, 'rb') as f:
        r = requests.post(BASE, data={
            'api_key': API_KEY,
            'facility_code': FACILITY,
        }, files={'image': f})
```

```

r.raise_for_status()

slug = r.json()['id']

# Polling – on attend `is_done: true`
for _ in range(60): # max 60 tentatives = ~3 minutes
    time.sleep(3)

    r = requests.get(BASE, params={'id': slug})

    data = r.json()

    if data.get('is_done') is True:
        if data.get('error_status'):
            raise RuntimeError(data.get('error_message') or data.get('message'))

        return data

    raise TimeoutError(f"Analyse {slug} non terminée après 3 minutes")

result = analyze('panoramique_test.jpg')
print(result)

```

Exemple JavaScript (Node.js / browser)

```

const API_KEY = 'VOTRE_CLE_API';

const FACILITY = 'VOTRE_CODE_ETABLISSEMENT';

const BASE = 'https://aiv4.thakaamed.com/api/v2.3/fr/analyze/radiography/';

async function analyze(imageFile) {
    const form = new FormData();

    form.append('api_key', API_KEY);
    form.append('facility_code', FACILITY);
    form.append('image', imageFile);

    const submit = await fetch(BASE, { method: 'POST', body: form });
    const { id: slug } = await submit.json();

    for (let i = 0; i < 60; i++) {

```

```

await new Promise(r => setTimeout(r, 3000));

const poll = await fetch(`${BASE}?id=${slug}`);

const data = await poll.json();

if (data.is_done === true) {

    if (data.error_status) throw new Error(data.error_message || data.message);

    return data;

}

}

throw new Error(`Timeout pour ${slug}`);
}

```

5. Anatomie d'une réponse

Quand votre GET réussit (**is_done: true**), vous recevez un JSON profond et richement structuré (~300 Ko à 1 Mo selon le nombre de détections). Champs **de premier niveau** :

Champ	Description
id	Le slug que vous avez reçu au POST.
is_done	true quand l'analyse est terminée. C'est votre signal de fin.
error_status	true en cas d'erreur côté serveur (image illisible, etc.).
message	Message lisible (ex. "Analysis completed successfully.").
error_message	Détail si error_status est vrai.
version	Version du moteur ayant produit le résultat.
response_time	Temps de calcul côté serveur (en secondes).
original_image	URL de l'image que vous avez uploadée (CDN ThakaaMed).
draw_image	 URL d'une image PNG annotée (overlays, bounding boxes dessinés) — prêt à afficher tel quel.
embeded_link	 URL d'un visualiseur web intégrable en <code><iframe></code> qui montre l'image + détections interactives.
embeded_report_link	URL d'un rapport PDF imprimable.
results	Objet (pas tableau !) contenant l'analyse structurée. Détaillé ci-dessous.

 **Astuce hackathon** : **draw_image** et **embeded_Link** sont des super-pouvoirs. Vous pouvez livrer une démo visuelle impressionnante en affichant simplement le **draw_image** (overlays déjà

dessinés) ou en intégrant `embeded_Link` dans une `<iframe>` — pas besoin de coder votre propre rendu canvas.

Structure de **results**

```
results: {  
  implant_brands: [],  
  tooth_results: {  
    "11": {                                     // ← clé = numérotation FDI (11-48)  
      slug: "...",  
      label_slug: "...",  
      is_missing: false,  
      illnesses: [                             // ← pathologies détectées sur cette dent  
        {  
          name: "Dental Pulp",  
          slug: "...",  
          probability: 42.39,    // ← échelle 0-100, PAS 0-1  
          icd_dict: { icd_code: "Z97.2", icd_desc: "Filling-Overhanging margin" }  
        }  
      ],  
      treatment_methods: [],  
      coordinates: {                       // ← bounding box  
        xmin: 1128.0, ymin: 526.0,  
        xmax: 1214.0, ymax: 768.0,  
        proba: 88.81              // ← confiance de la dent (0-100)  
      },  
      list_coordinates: [[x,y], [x,y], ...] // polygone exact  
    },  
    "12": { ... },  
    ...  
  }  
}
```

Points importants :

- Les **coordinates** utilisent les clés **xmin/ymin/xmax/ymax** (pas **x1/y1/x2/y2**) avec un champ **proba** 0–100.
- Les probabilités d'**illnesses** sont aussi sur l'échelle 0–100.
- **tooth_results** est un **dictionnaire** indexé par numéro FDI ("**11**", "**12**", ..., "**48**"), pas une liste.
- Sur une panoramique typique vous obtenez ~32 dents avec ~30–50 pathologies au total.

 **Astuce** : un échantillon réel complet est dans `examples/sample_v23_analysis.json` (~1,3 Mo). Ouvrez-le dans VSCode ou explorez avec `jq` pour cartographier la structure avant de brûler des jetons.

6. Quotas, limites, et bonnes pratiques

Ressource	Limite par équipe
Jetons	1 000 analyses (1 jeton consommé par analyse, peu importe le type d'image)
Durée de vie	30 jours (expiration le 2026-06-08)
Taille d'image max	~50 Mo par fichier (recommandé : <10 Mo, JPEG/PNG)
Requêtes simultanées	Pas de limite stricte, mais soyez raisonnables — l'infrastructure est partagée entre toutes les équipes
Format d'image	JPEG, PNG. DICOM non supporté directement — convertissez côté client

Si vous épuisez vos jetons

Contactez les organisateurs. Une recharge de **+500 jetons** peut être accordée à mi-parcours en cas de besoin justifié (par exemple, vous construisez un pipeline batch et avez besoin de tester sur 200 images).

Confidentialité — IMPORTANT

 **N'envoyez aucune donnée patient réelle sans consentement explicite.** Pour le hackathon, utilisez :

- Des **datasets publics** de radiographies dentaires (par exemple sur Kaggle, GitHub, Hugging Face) ;
- Des images **synthétiques** ou **anonymisées** que les organisateurs vous fourniront ;
- Vos propres radiographies si un membre de l'équipe est dentiste et a accès à ses propres images.

L'API n'est **pas un dispositif médical certifié** dans ce contexte. Vos prototypes sont des démos de hackathon, pas des outils cliniques. Mentionnez cela clairement dans toute interface utilisateur que vous construirez.

7. Idées de défis 🚀

Voici **12 idées de projets** que vous pouvez relever (ou qui peuvent vous inspirer pour les vôtres). Choisissez celle qui résonne avec votre équipe — ou combinez-en plusieurs. Chaque idée est conçue pour être réalisable en 48–72 heures par une équipe de 3–5 personnes.

🔊 Défi 1 — L'Explicateur pour Patients

« Madame Dupont, voici votre radio. L'IA a trouvé 3 caries et un implant en bonne santé. Voulez-vous que je vous explique chacune ? »

Construisez une application web ou mobile qui :

- Prend une radiographie en entrée ;
- Affiche le résultat en **langage accessible** (pas de jargon dentaire) ;
- Génère une **explication audio** (text-to-speech) pour les patients qui préfèrent l'écoute ;
- Propose une version **multilingue** (français, arabe, anglais — utile en cabinet multi-culturel) ;
- Inclut des **illustrations vulgarisées** (ex. : « voici à quoi ressemble une carie, voici comment elle progresse si non traitée »).

Bonus : intégrez un LLM (Claude, GPT, Mistral) qui répond aux questions du patient sur sa radio en temps réel (« et cette tache là, c'est quoi ? »).

😞 Défi 2 — Le Deuxième Avis IA

« Mon dentiste m'a dit qu'il fallait 3 couronnes. Je veux un deuxième avis. »

L'utilisateur uploade :

1. Sa radiographie ;
2. Le **plan de traitement proposé par son dentiste** (texte libre).

Votre app :

- Compare les détections de l'IA avec le plan proposé ;
- Identifie les **traitements suggérés sans détection IA correspondante** (potentielle sur-prescription) ;
- Identifie les **détections IA non couvertes** par le plan (potentielle sous-prescription) ;
- Présente un **rapport de divergence** clair, avec un score de confiance.

Bonus : interfacez avec un LLM pour générer un résumé en langage naturel et des questions à poser à son dentiste.

🚑 Défi 3 — Le Triagiste d'Urgence

« 47 patients en salle d'attente. Qui voir en premier ? »

Pour les cliniques surchargées (ONG, missions humanitaires, pays à faible densité de dentistes) :

- Interface caméra simple → prendre une photo de la radio sur l'écran ou capturer directement ;
- L'IA analyse en temps réel ;
- Calcule un **score d'urgence** (ex. : abcès péri-apical actif > carie profonde > carie superficielle > détartrage) ;

- Trie automatiquement la file d'attente et alerte le dentiste sur les cas critiques.

Bonus : mode hors-ligne avec mise en cache des analyses pour zones à connexion limitée.

Défi 4 — L'Anti-Anxiété pour Enfants

« N'aie pas peur, M. Carie est juste un petit monstre — regarde, le dentiste va le chasser ! »

Transformez l'expérience anxiogène de la radio en jeu :

- L'enfant uploade sa radio (ou ses parents le font) ;
- L'app **transforme la radio noir et blanc en illustration colorée et amicale** ;
- Chaque carie devient un **personnage animé** (« M. Carie », « Princesse Plombage », etc.) ;
- Une histoire interactive raconte ce qui va se passer chez le dentiste ;
- Système de **récompenses** pour le brossage post-visite.

Bonus : génération d'images via Stable Diffusion ou DALL-E à partir des coordonnées des détections.

Défi 5 — Le Compagnon de l'Étudiant en Dentaire

« Je suis en 4ème année. Je veux m'entraîner à lire des radios. »

Plateforme d'apprentissage gamifiée :

- Bibliothèque de radiographies (datasets publics) ;
- L'étudiant **annote la radio** (cliquer sur les caries, restaurations, etc.) ;
- L'IA compare ses annotations à celles du modèle ThakaaMed ;
- **Score** par exercice, **classement** entre étudiants, **statistiques** par type de pathologie ;
- Mode « cas cliniques » : 10 radios par session, débrief détaillé en fin.

Bonus : créer un assistant LLM qui agit comme un tuteur (« pourquoi as-tu raté cette carie ? regarde la zone X... »).

Défi 6 — Le Carnet de Santé Bucco-Dentaire

« J'ai mes radios des 5 dernières années. Mon état empire-t-il ? »

Application personnelle (web ou mobile) :

- L'utilisateur uploade plusieurs radios prises à des dates différentes ;
- L'IA analyse chacune ;
- L'app **détecte les évolutions** : nouvelle carie, traitement réussi, perte osseuse, etc. ;
- Visualisation **timeline** avec courbes de santé bucco-dentaire ;
- Recommandations personnalisées (« vous avez 3 nouvelles caries en 1 an — augmentez vos visites à 2x/an »).

Bonus : prédiction par modèle ML simple — « si vous ne traitez pas, voici où vous serez dans 2 ans ».

Défi 7 — L'Assistant Téléconsultation

« Le village le plus proche avec un dentiste est à 200 km. »

Pour les déserts médicaux :

- Une infirmière ou un médecin généraliste prend la radio dans une clinique de proximité ;
- L'IA fait une **pré-analyse automatique** ;
- Un **rapport pré-rempli** est généré et envoyé à un dentiste spécialiste à distance ;
- Le spécialiste valide / modifie / commente ;
- Le rapport final est renvoyé au patient en 24h.

Bonus : queue intelligente — les cas les plus urgents (selon l'IA) montent en haut de la liste du spécialiste.

Défi 8 — L'Estimateur de Coûts et de Pré-Autorisation

« Combien va me coûter ce qu'on m'a proposé ? »

App de transparence financière :

- Upload de la radio + ville/pays de résidence ;
- L'IA détecte les traitements probablement nécessaires ;
- L'app croise avec une **base de données de tarifs** (publique ou que vous construisez à la main) ;
- Génère une **fourchette de prix** par traitement (« plombage molaire : 80–150 € selon la clinique ») ;
- Génère une **lettre pré-formatée pour la mutuelle / l'assurance** avec les codes CCAM/ICD-10 correspondants.

Bonus : intégration avec les API de comparateurs de prix dentaires existants.

Défi 9 — Le Chatbot Dentaire Intelligent (RAG sur radios)

« Voici ma radio. Dis-moi tout. »

Combinez le moteur ThakaaMed avec un LLM moderne :

- L'utilisateur upload sa radio ;
- L'IA ThakaaMed produit le JSON d'analyse ;
- Vous **injectez ce JSON dans le contexte d'un LLM** (Claude, GPT, Mistral) ;
- L'utilisateur peut **discuter** : « explique-moi cette détection », « est-ce grave ? », « combien de temps avant que ça empire ? », « quelles questions poser à mon dentiste ? ».

Bonus : ajoutez un mode « cite tes sources » où le LLM lie chaque affirmation à un détecteur précis du JSON (« d'après le détecteur de carie occlusale niveau 2, dent 16, confiance 0.91... »).

Défi 10 — La Visualisation 3D / Réalité Augmentée

« Wow, c'est trop cool. »

Construisez quelque chose de spectaculaire visuellement :

- À partir des coordonnées polygonales de l'API, générez un **modèle 3D approximatif** des dents ;
- Mode AR (WebXR ou app mobile) : la caméra du téléphone affiche les détections **superposées sur le visage** du patient en temps réel ;
- Animations : montrer la carie qui « creuse » la dent, l'implant qui s'intègre dans l'os, etc.

Bonus : mode VR pour casques immersifs — l'utilisateur se promène à l'intérieur de sa propre bouche.

Défi 11 — La Cartographie Épidémiologique

« Quel pourcentage de la population a des caries non traitées dans cette région ? »

Outil de santé publique :

- Importez des datasets publics anonymisés de radios par zone géographique ;
- Lancez l'IA sur un grand nombre d'images (pipeline batch — utilisez vos 1 000 jetons stratégiquement !) ;
- Construisez une **carte interactive** des pathologies dentaires par région ;
- Identifiez des **patterns** : zones avec forte incidence de caries ? d'implants ? de pertes osseuses ?

Bonus : exportez vos résultats sous forme de rapport pour des ONG ou des ministères de santé publique.

Défi 12 — Surprenez-nous !

Vous avez une idée à laquelle personne n'a pensé ? Allez-y. Quelques pistes très libres :

-  **Jeu vidéo** où le héros nettoie des dents géantes (les caries sont des ennemis) ;
-  **Génération musicale** : chaque dent est une note, chaque carie est une dissonance — jouez votre propre radio en symphonie ;
-  **Détection croisée vétérinaire** : tester si nos modèles fonctionnent (ou pas) sur des radios canines/félines, et publier le rapport ;
-  **Génération automatique de rapports légaux** pour les assurances, avec mise en forme PDF professionnelle ;
-  **Alerte familiale** : « papa, ta dent 26 a empiré depuis l'an dernier, prends rendez-vous » — push notifications intelligentes ;
-  **Outil de R&D dentaire** : statistiques agrégées sur des centaines de radios pour tester une hypothèse clinique.

La règle d'or : ayez du fun, soyez ambitieux, et construisez quelque chose dont vous serez fiers.

8. Conseils pratiques pour démarrer

Les 30 premières minutes

3. **Faites un appel test immédiatement** avec une radio d'exemple. Sauvegardez la réponse JSON dans un fichier. Explorez sa structure dans un éditeur de JSON (VSCode, jq, Postman).
4. **Validez votre `facility\_code`** — sans lui, l'API v2.3 retourne une erreur 400. C'est la cause #1 de blocage en début de hackathon.
5. **Mettez en place le polling tôt** — l'erreur classique est de faire un GET trop tôt et de croire que l'API est cassée. Attendez 3–5 secondes minimum.
6. **Cachez vos résultats** — chaque appel coûte 1 jeton. En dev, sauvegardez les JSON localement et travaillez dessus offline pour ne pas brûler vos jetons.

Erreurs fréquentes

Erreur	Cause probable	Solution
400 Bad Request – Facility key required	<code>facility_code</code> manquant ou vide	Ajoutez le champ dans votre form-data
404 / wrong API key	Faute de frappe dans <code>api_key</code>	Copier-coller exact, attention aux espaces
Insufficient tokens	Vous avez épuisé vos 1 000 jetons	Contactez les organisateurs pour une recharge
Image too large	Fichier > 50 Mo	Compressez ou redimensionnez côté client
Réponse vide même après 1 minute	Image corrompue ou format non supporté	Vérifiez le fichier avec un viewer avant upload

Gestion de votre quota de 1 000 jetons

- **Phase 1 (exploration, jours 1)** : ~50 appels max. Apprenez la structure de la réponse, testez 3–4 types de radios différents.
- **Phase 2 (développement, jours 1–2)** : ~200 appels. Cachez agressivement vos résultats. Réutilisez les JSON sauvegardés.
- **Phase 3 (démonstration et vidéo finale)** : gardez ~100–200 jetons pour générer de belles démos sur de vraies images.
- **Réserve** : ~500 jetons pour les imprévus. Croyez-nous, vous en aurez besoin.

Outils recommandés

- **Postman** ou **Insomnia** pour explorer l'API manuellement avant de coder.
- **jq** en ligne de commande pour naviguer dans le JSON de réponse.
- **Streamlit** ou **Gradio** (Python) pour des prototypes UI ultra-rapides.
- **Next.js** ou **Vite + React** pour quelque chose de plus polish.
- **Claude API** ou **OpenAI API** pour la couche LLM/conversationnelle (si vos défis l'exigent).

9. Datasets de radiographies (pour tester sans patients réels)

Quelques sources publiques pour vous fournir en images de test :

Source	Type	Lien
Kaggle — Dental Panoramic X-rays	Panoramiques	Recherchez « dental panoramic » sur kaggle.com
Hugging Face — Dental Datasets	Mixte	huggingface.co/datasets (chercher « dental »)
GitHub — open-dental-datasets	Mixte	Recherche GitHub sur « dental radiography dataset »
Synthetic data	Tous types	Demandez aux organisateurs

Si vous avez un membre dentiste ou étudiant en dentaire dans l'équipe, il peut souvent fournir des images d'entraînement de son école — vérifiez juste que c'est anonymisé.

10. Critères d'évaluation (suggérés)

Le jury évaluera probablement votre projet sur :

Critère	Pondération suggérée
Impact réel — votre projet résout-il un vrai problème dentaire ?	30%
Qualité technique — code propre, intégration solide de l'API, gestion des erreurs	25%
Créativité et originalité — avez-vous fait quelque chose de surprenant ?	20%
Polish et UX — l'expérience utilisateur est-elle agréable ?	15%
Présentation et démo — savez-vous raconter votre projet ?	10%

*Conseil bonus : enregistrez une **vidéo de démo** de 2 minutes au minimum. Beaucoup de bons projets se font dépasser à cause d'une mauvaise présentation.*

11. Foire aux questions

Q : L'API fonctionne-t-elle pour les radios dentaires d'animaux (chiens, chats) ? R : Les modèles ont été entraînés exclusivement sur des radios humaines. Vous pouvez tester par curiosité, mais les résultats ne seront pas fiables. (Cela dit, c'est un défi intéressant en soi — voir Défi 12.)

Q : Puis-je utiliser un LLM (Claude, GPT) en plus de l'API ThakaaMed ? R : Absolument oui — c'est même fortement encouragé. Vos clés ThakaaMed ne couvrent que ThakaaMed ; les LLMs nécessitent vos propres clés (créez un compte gratuit sur Anthropic, OpenAI ou Mistral).

Q : Mes 1 000 jetons sont-ils partagés entre les membres de mon équipe ? R : Oui. Tous les appels avec votre `api_key` puisent dans le même pool de 1 000.

Q : Puis-je sauvegarder les réponses JSON et travailler offline ? R : Oui, c'est même la meilleure pratique pour économiser vos jetons. Téléchargez 20–30 JSON de référence en début de hackathon et utilisez-les en dev.

Q : Que se passe-t-il après le 2026-06-08 ? R : Votre clé devient inactive. L'API renverra une erreur de quota. Si vous voulez continuer votre projet après le hackathon, contactez ThakaaMed pour une licence commerciale.

Q : L'API renvoie-t-elle des codes ICD-10 ou CCAM ? R : Le mapping ICD-10 existe mais n'est pas activé par défaut sur vos clés (pour rester simple). Si votre projet en a besoin (par exemple Défi 8), demandez aux organisateurs d'activer le flag `icd_status` sur votre compte — c'est gratuit et rapide.

Q : Le serveur est lent / mes appels échouent — qu'est-ce qui se passe ? R : L'infrastructure est partagée entre les 12 équipes. Si tout le monde lance des batchs en même temps, ça peut ralentir. Soyez sympas, étalez vos appels. En cas de panne suspectée, contactez les organisateurs.

12. Support et contact

- **Coordinateur du hackathon** : AABIL Rime - +212708150051 - aabil.rime@inpt.ac.ma

En cas de problème technique avec l'API (réponses inattendues, erreurs persistantes, jetons à recharger), contactez d'abord le coordinateur du hackathon — il fait le pont avec l'équipe ThakaaMed.

13. Disclaimer médical et éthique

 **Lecture obligatoire avant la démo finale.**

Les modèles ThakaaMed AI utilisés ici sont des outils d'aide au diagnostic conçus pour des dentistes professionnels exerçant en cabinet. **Aucun prototype construit pendant ce hackathon ne doit être utilisé directement sur des patients sans validation par un dentiste qualifié.**

Si votre interface utilisateur cible le grand public (patients), incluez **systématiquement** :

- Une mention claire «  Cet outil est un prototype de démonstration. Il ne remplace pas une consultation chez un dentiste qualifié. »
- Pas de promesse de diagnostic définitif.
- Pas de prescription de traitement.

Respectez la confidentialité : ne stockez aucune image patient sans consentement écrit, et anonymisez tout ce que vous présentez en public (vidéo de démo, captures d'écran, etc.).

14. Pour aller plus loin

Une fois le hackathon terminé, si votre projet a du potentiel :

- ThakaaMed propose des **licences commerciales** pour intégrer le moteur dans des produits réels.
- Plusieurs des défis ci-dessus correspondent à des **opportunités business actuelles** dans le secteur de la santé dentaire numérique — n'hésitez pas à transformer votre prototype en startup.
- Restez en contact avec l'équipe ThakaaMed via les coordinateurs du hackathon.

Bonne chance à toutes les équipes ! 🦷⚡

Que le meilleur projet gagne — et surtout, que vous appreniez et vous amusiez. C'est la priorité numéro un.